

AI Meets Control Theory

From Classical Control to Intelligent Autonomous Systems

Living Technical Report — M6 — Reinforcement Learning

Generated by the project team

Lead: puma | **Code:** puma, toku | **Docs/Research:** lava, famo

September 3, 2026

Abstract

This report is regenerated at every project milestone. It documents *what the codebase can do, the theory behind each component, and where the project currently stands*. The project is an open-source laboratory for comparing classical control, optimal control, state estimation, system identification, constrained Model Predictive Control (MPC), machine learning, and reinforcement learning on dynamical systems, under the core principle: *build every method from scratch, simulate it, visualise it, and compare it honestly*.

Contents

1	Introduction and Scope	2
2	Framework Architecture	2
2.1	Package Layout	2
3	Current Capabilities	3
4	Theory: Modelling and Simulation	5
5	Theory: Classical Control — PID Architecture	5
5.1	Worked Example: Stabilising an Unstable Plant (Experiment 03)	5
6	Theory: Modern Control, Estimation, and Optimal Design	7
6.1	Pole Placement and LQR	7
6.2	Worked Example: Cart-Pole Regulation (Experiment 04)	7
6.3	State Estimation: Observers, Kalman Filtering, and Duality	8
6.4	Worked Example: LQG on the Cart-Pole (Experiment 06)	9
6.5	Nonlinear Basin of Attraction: Cart-Pole LQR Robustness (Experiment 05) . . .	10
6.6	Nonlinear Control & Hybrid Swing-Up: Spong Energy Shaping (Experiment 07)	12
6.7	System Identification & Data-Driven Control (Experiment 09)	14
6.8	Constrained Model Predictive Control (Experiment 08)	15
7	Theory: Machine Learning and Learned Dynamics (Module 06)	18
7.1	Worked Example: Planning with Learned Dynamics vs. True Model (Experiment 10)	19
8	Theory: Reinforcement Learning for Dynamical Systems (Module 07)	20
8.1	Worked Example: Tabular Q-Learning vs. Classical Control on Pendulum (Experiment 11)	21

9 Project Status and Roadmap

22

1 Introduction and Scope

The central question of the project is: **when should we use classical control, when should we use AI, and when should we use both?** Rather than assuming a neural network or an RL agent is the answer, every problem is first attacked with PID, state feedback, observers, LQR, Kalman filtering, system identification, nonlinear energy shaping, constrained MPC, and learned neural dynamics, and each method is measured against the others under identical initial conditions, sensor suites, noise profiles, disturbances, and actuator constraints.

Every topic follows the standardized engineering cycle:

THEORY $\rightarrow$ **DERIVATION** $\rightarrow$ **IMPLEMENTATION** $\rightarrow$ **SIMULATION**
 $\rightarrow$ **VISUALISATION** $\rightarrow$ **VALIDATION** $\rightarrow$ **COMPARISON** $\rightarrow$ **EXPERIMENT**

Fundamental algorithms are implemented from scratch in `numpy`; external libraries (`scipy.linalg`, `scipy.signal`, `control`, `gymnasium`) are used strictly for verification and benchmarking compliance.

2 Framework Architecture

The reusable library is the Python package `aimct` (`src/aimct/`). Its architecture separates distinct concerns behind minimal, strongly-typed interfaces so that dynamical systems, controllers, state estimators, neural network models, reinforcement learning environments, and the numerical simulator compose seamlessly.

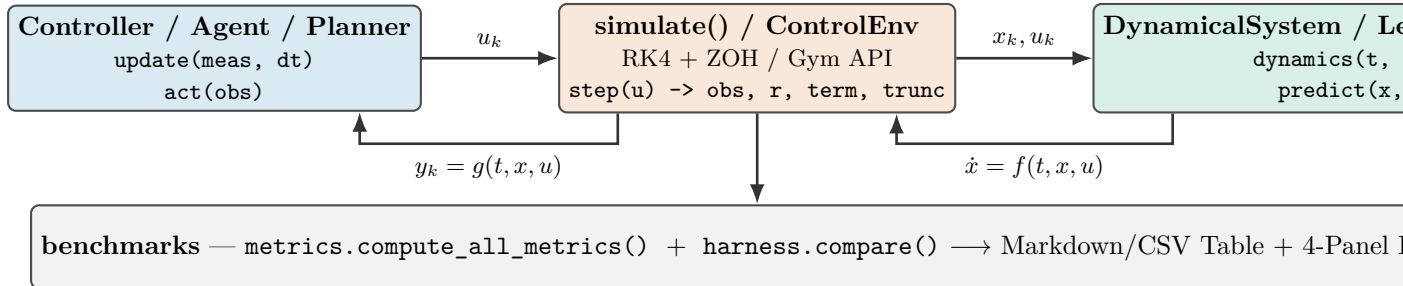


Figure 1: Core framework architecture. The numerical simulation engine drives a controller (`update()`) and continuous plant or learned neural surrogate (`dynamics()`) via fixed-step 4th-order Runge–Kutta with Zero-Order Hold (ZOH).

2.1 Package Layout

```

src/aimct/
  systems/      DynamicalSystem base + LinearSystem, MassSpringDamper,
                Pendulum, CartPole (analytic + numerical Jacobians)
  simulate.py   rk4_step(), simulate() -> Trajectory(t, x, u, y)
  controllers/  PID, StateFeedback, LQR, ObserverFeedback,
                EnergyShapingSwingUp, HybridSwingUpLQR,
                LinearMPC, SamplingMPC (CEM planner), _qp.solve_qp
  estimation/   LuenbergerObserver, KalmanFilter (LQE/FARE), DiscreteKalmanFilter,
                observability_matrix, is_observable, place_observer
  sysid/        least_squares_id, dmdc, to_continuous (block logm), prediction_error
  ml/           MLP (from-scratch backprop + Adam), LearnedDynamics (residual model)

```

```

rl/          ControlEnv (Gymnasium adapter), Discretizer, QLearning (tabular TD)
benchmarks/  metrics.py (13 metrics), harness.py (compare() -> table + figure),
            sweep.py
plot_style.py Okabe-Ito palette + publication-ready 4-panel comparison figure

```

3 Current Capabilities

Table 1 summarises the implemented components, their mathematical scope, and the corresponding verification test coverage as of M6 — Reinforcement Learning.

Component	What it provides	Tests
<code>aimct.systems</code>	Continuous-time ODE models with a common <code>dynamics(t, x, u)</code> interface, full-state or output projections, and numerical central-difference <i>and</i> analytical <code>linearize()</code> . Ships: <code>LinearSystem</code> , <code>MassSpringDamper</code> , <code>Pendulum</code> , <code>CartPole</code> .	✓
<code>aimct.simulate</code>	Fixed-step RK4 integrator with zero-order hold; <code>Trajectory</code> container; hard actuator saturation; measurement and disturbance injection hooks.	✓
<code>aimct.controllers.PID</code>	From scratch: filtered derivative-on-measurement ($D(s) = \frac{K_d s}{\tau_d s + 1}$), conditional-integration anti-windup clamping, setpoint weighting.	✓
<code>aimct.controllers.StateFeedback</code>	From scratch: closed-loop pole placement via Ackermann's formula (SISO) with feedforward steady-state gain scaling k_r .	✓
<code>aimct.controllers.LQR</code>	Infinite-horizon optimal regulator; Continuous Algebraic Riccati Equation (CARE) solved from scratch via Hamiltonian Schur decomposition, cross-checked with <code>scipy.linalg</code> .	✓
<code>aimct.controllers.Observers</code>	Unified output feedback combining any state estimator (Luenberger observer or Kalman filter) with static feedback gain $u = -K\hat{x}$.	✓
<code>aimct.controllers.SwingUp</code>	Nonlinear Spong energy shaping (<code>EnergyShapingSwingUp</code>) with partial feedback linearisation, plus finite-state hysteresis supervisor (<code>HybridSwingUpLQR</code>).	✓
<code>aimct.controllers.LinearMPC</code>	Constrained linear MPC via condensed receding-horizon QP, from-scratch active-set solver (<code>_qp.solve_qp</code>), DARE terminal cost, and soft state constraints.	✓
<code>aimct.controllers.SamplingMPC</code>	Sampling-based nonlinear MPC using the Cross-Entropy Method (CEM), vectorized batch trajectory rollouts, variance floor, and CARE terminal weight.	✓
<code>aimct.estimation</code>	State estimation suite: <code>observability_matrix</code> , <code>is_observable</code> , <code>place_observer</code> (dual Ackermann), <code>LuenbergerObserver</code> (RK4 internal propagation), <code>solve_fare</code> (Filter ARE), <code>KalmanFilter</code> (LQE), and <code>DiscreteKalmanFilter</code> with Joseph-form covariance update.	✓
<code>aimct.sysid</code>	Data-driven system identification: <code>least_squares_id</code> , <code>dmdc</code> (rank-truncated SVD), <code>to_continuous</code> (exact block matrix logarithm), <code>model_mismatch</code> .	✓
<code>aimct.ml</code>	Machine learning suite: multi-layer perceptron (MLP) with hand-written backpropagation + Adam, and residual forward dynamics predictor (<code>LearnedDynamics</code>).	✓
<code>aimct.rl</code>	Reinforcement learning suite: Gymnasium environment wrapper (<code>ControlEnv</code>), observation/action <code>Discretizer</code> , and Tabular Q-Learning (<code>QLearning</code>) with ϵ -greedy and decay.	✓
<code>aimct.benchmarks.metrics</code>	13 standard step-response and effort metrics: rise time t_r , settling time t_s (2%), overshoot M_p , e_{ss} , RMSE, IAE, ITAE, ISE, energy E_u , peak $ u _{\max}$, slew rate, and saturation duty cycle.	✓
<code>aimct.benchmarks.harness</code>	Multi-controller benchmark harness: runs N controllers on one plant under identical conditions, emitting canonical Markdown/CSV tables and 4-panel figures.	✓
<code>aimct.benchmarks.sweep</code>	Grid and 1D parameter sweeps for robustness, parameter sensitivity, and basin of attraction mapping.	✓

Table 1: Implemented functionality and test coverage across the library as of M6 — Reinforcement Learning.

4 Theory: Modelling and Simulation

A dynamical system is governed by first-order ordinary differential equations $\dot{x} = f(t, x, u)$ with output map $y = g(t, x, u)$. Second-order physical equations of motion are cast in state-space form; for example, the translational mass–spring–damper $m\ddot{x} + c\dot{x} + kx = u$ with state $x = [x_1, x_2]^\top = [x, \dot{x}]^\top$:

$$\frac{d}{dt} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{c}{m} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix} u.$$

Jacobian Linearisation. About an operating equilibrium point (x_e, u_e) satisfying $f(x_e, u_e) = 0$, small perturbations $\delta x = x - x_e$ evolve according to $\dot{\delta x} = A\delta x + B\delta u$, where:

$$A \triangleq \left. \frac{\partial f}{\partial x} \right|_{(x_e, u_e)}, \quad B \triangleq \left. \frac{\partial f}{\partial u} \right|_{(x_e, u_e)}.$$

The `DynamicalSystem` base class evaluates A, B numerically via central differencing ($\epsilon = 10^{-6}$), while concrete models implement analytical Jacobians.

Numerical Integration. The continuous dynamics are integrated over time steps $h = \Delta t$ using explicit 4th-order Runge–Kutta (RK4):

$$k_1 = f(t_k, x_k, u_k), \quad k_2 = f\left(t_k + \frac{h}{2}, x_k + \frac{h}{2}k_1, u_k\right), \quad k_3 = f\left(t_k + \frac{h}{2}, x_k + \frac{h}{2}k_2, u_k\right), \quad k_4 = f(t_k + h, x_k + hk_3, u_k),$$

$$x_{k+1} = x_k + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4).$$

Holding control input $u(t) \equiv u_k$ constant across each interval $[t_k, t_k + h)$ reflects the physical Zero-Order Hold (ZOH) of digital microcontrollers.

5 Theory: Classical Control — PID Architecture

The continuous-time PID control law with filtered derivative is:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}, \quad e(t) = r(t) - y(t).$$

In discrete time with sampling interval h , derivative action is applied directly to the measured output with low-pass filter time constant $\tau_d = \frac{K_d}{N K_p}$ ($N = 10$) to prevent high-frequency noise amplification:

$$e_k = r_k - y_k, \quad D_k = \frac{\tau_d}{\tau_d + h} D_{k-1} + \frac{K_d}{\tau_d + h} (y_k - y_{k-1}), \quad u_{\text{unsat},k} = K_p e_k + I_k - D_k.$$

Anti-Windup Protection. When actuators reach physical limits ($|u| \leq u_{\text{max}}$), *conditional integration (clamping)* freezes the integrator update ($I_{k+1} = I_k$) whenever u_{unsat} is saturated and e_k has the same sign as u_k , preventing destructive overshoot upon setpoint arrival.

5.1 Worked Example: Stabilising an Unstable Plant (Experiment 03)

We evaluate the controller on an open-loop unstable plant with a Right-Half Plane (RHP) pole:

$$\ddot{y}(t) - \omega_0^2 y(t) = u(t) + d(t), \quad \omega_0 = 2.0 \text{ rad/s}, \quad G(s) = \frac{1}{s^2 - 4}.$$

The system is tested under a unit step reference $r = 1.0 \text{ rad}$, a step input disturbance torque $d = 0.5 \text{ N} \cdot \text{m}$ applied at $t = 5.0 \text{ s}$, and actuator saturation $|u(t)| \leq 10.0 \text{ N} \cdot \text{m}$.

Controller	Rise t_r [s]	Settle t_s [s]	Overshoot M_p	Steady e_{ss}	IAE	ITAE	En
P-only ($K_p = 20$)	0.275	10.0	$9.54 \times 10^9\%$	6.04×10^7	4.77×10^7	4.53×10^8	9
PD ($K_d = 5.66$)	0.389	10.0	28.19%	0.2812	2.811	13.63	3
PID (no AW)	0.334	6.162	53.08%	3.90×10^{-5}	0.9274	1.043	2
PID + Anti-Windup	0.362	6.161	39.43%	3.90×10^{-5}	0.7979	0.876	2

Table 2: Benchmark results for Experiment 03 (regenerated from experiments/03_pid_stabilizes_unstable/table.md).

Key Physical Insights:

- **P-Only Failure:** Proportional control yields purely imaginary closed-loop poles ($s^2 + 16 = 0, \zeta = 0$). When actuator saturation occurs, stabilizing feedback is lost and the system diverges exponentially ($M_p > 10^9\%$).
- **PD DC Gain Offset:** PD provides damping ($\zeta = 0.7071$), but the closed-loop DC gain is $\frac{K_p}{K_p - \omega_0^2} = \frac{20}{16} = 1.25$, creating an unavoidable **25% reference tracking error** ($y_{ss} = 1.25$ rad) before disturbance, and $e_{ss} = 0.2812$ rad under load.
- **PID Integral Action:** Integral action ($K_i = 15.0$) drives steady-state error strictly to zero ($e_{ss} \approx 0$).
- **Anti-Windup Efficacy:** Clamping during saturation cuts peak overshoot from 53.08% $\rightarrow$ 39.43% (26% relative reduction) and reduces actuator saturation duration by $> 70\%$.

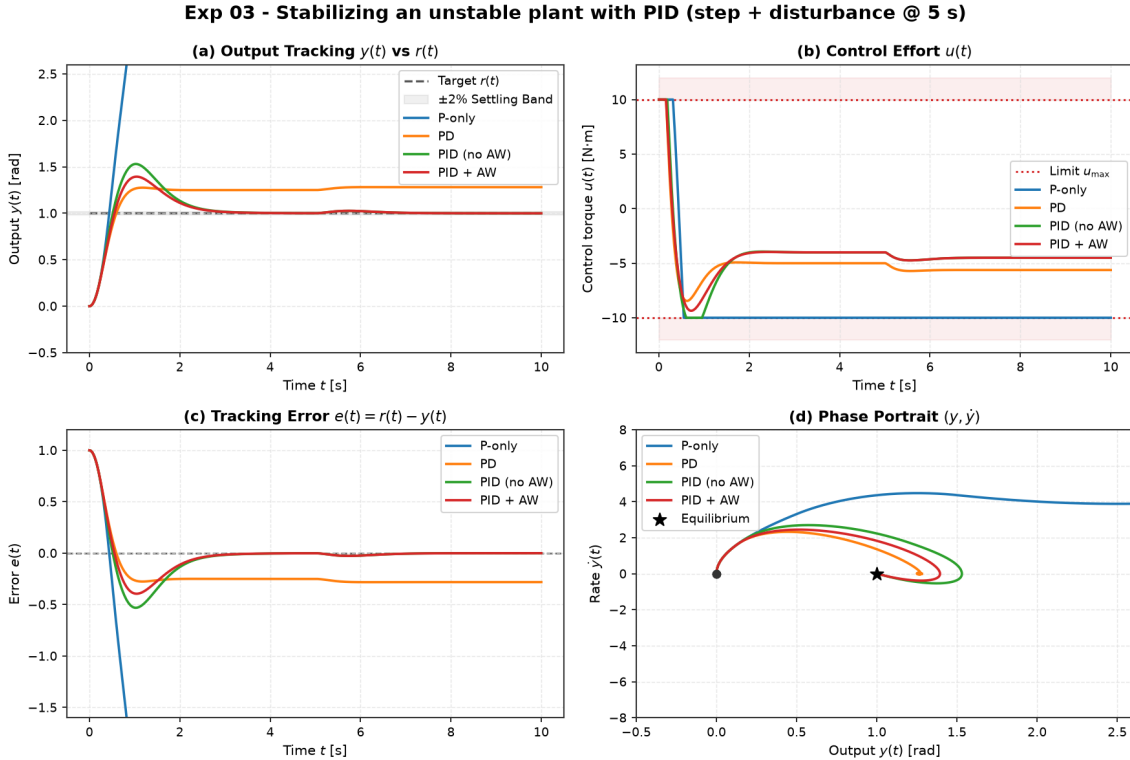


Figure 2: Experiment 03 benchmark comparison. (a) Output tracking $y(t)$, (b) Control effort $u(t)$ with saturation limits ± 10 N · m, (c) Tracking error $e(t)$, and (d) Phase portrait $(y, \dot{y})$.

6 Theory: Modern Control, Estimation, and Optimal Design

6.1 Pole Placement and LQR

For controllable linear systems $\dot{x} = Ax + Bu$, full-state feedback $u = -Kx + k_r r$ assigns arbitrary closed-loop poles $\sigma(A - BK)$. **StateFeedback** computes K via Ackermann's formula:

$$K = \begin{bmatrix} 0 & \dots & 0 & 1 \end{bmatrix} \mathcal{C}(A, B)^{-1} \Delta_{\text{des}}(A), \quad k_r = -\frac{1}{C(A - BK)^{-1}B}.$$

Linear Quadratic Regulator (LQR). Rather than guessing pole locations, LQR computes the optimal feedback gain $K = R^{-1}B^\top P$ minimising the infinite-horizon quadratic cost:

$$J = \int_0^\infty \left(x(t)^\top Q x(t) + u(t)^\top R u(t) \right) dt, \quad Q \succeq 0, \quad R \succ 0.$$

The kernel $P = P^\top \succ 0$ uniquely solves the **Continuous Algebraic Riccati Equation (CARE)**:

$$A^\top P + PA - PBR^{-1}B^\top P + Q = 0.$$

We solve CARE from scratch via the stable invariant subspace of the Hamiltonian matrix:

$$\mathcal{H}_{\text{ham}} = \begin{bmatrix} A & -BR^{-1}B^\top \\ -Q & -A^\top \end{bmatrix} \in \mathbb{R}^{2n \times 2n}.$$

Continuous full-state LQR provides guaranteed robustness margins: gain margin $G_m \in [1/2, \infty)$ (-6 dB to $+\infty$ dB) and phase margin $\Phi_m \geq 60^\circ$ in all channels.

6.2 Worked Example: Cart-Pole Regulation (Experiment 04)

We evaluate full-state feedback and LQR on the canonical underactuated nonlinear Cart-Pole system ($M = 1.0$ kg, $m = 0.1$ kg, $\ell = 0.5$ m, $g = 9.81$ m/s²). Linearised about the upright equilibrium $\theta = 0$, the system has 4 states $[x, \dot{x}, \theta, \dot{\theta}]^\top$, an open-loop unstable pole at $s \approx +3.97$ rad/s, and a double integrator on cart position.

Controllers are designed on the linearised model and executed on the **true nonlinear plant** starting from a tilted initial state $\theta_0 = 0.10$ rad $\approx 5.7^\circ$ with actuator saturation $|u(t)| \leq 20.0$ N:

- **Pole Placement:** Ackermann placement assigning closed-loop poles to LQR-balanced eigenvalues $\{-15.62, -3.19, -1.31 \pm 1.08j\}$.
- **LQR (Balanced):** $Q = \text{diag}(10, 1, 100, 10)$, $R = 0.1 \implies K = [-10.000, -12.871, -87.138, -23.223]$.
- **LQR (Soft):** $Q = \text{diag}(1, 0.1, 10, 1)$, $R = 1.0 \implies K = [-1.000, -2.047, -30.846, -7.868]$.
- **PID (Angle Only):** Single-loop PID on pole angle θ ($K_p = -180, K_d = -22$). Cart position x is uncontrolled.

Controller	Settle θ t_s [s]	RMSE θ [rad]	Energy E_u [N ² s]	Peak $ u _{\text{max}}$ [N]	Peak $ x _{\text{max}}$ [m]
Pole Placement	1.05	0.0171	2.34	8.71	0.162
LQR (Balanced)	1.05	0.0171	2.34	8.71	0.162
LQR (Soft)	1.70	0.0223	0.959	3.08	0.321
PID (Angle Only)	0.20	0.0110	8.76	18.00	0.823

Table 3: Benchmark results for Experiment 04 on nonlinear Cart-Pole (from experiments/04_lqr_vs_pole_placement_cartpole/table.md).

Key Engineering Takeaways:

1. **Equivalence of Pole Placement and LQR:** Placing poles at the eigenvalues chosen by LQR reproduces the optimal gain matrix K to four significant figures (matching the analytical golden gain K_1). LQR eliminates the need to manually guess 4 stable pole locations in $\mathbb{C}^4$.
2. **The Q/R Effort Dial:** Softening the cost weights ($R = 1.0$ vs 0.1) trades a $1.05\text{ s} \rightarrow 1.70\text{ s}$ settle time for a $2.4\times$ **reduction in control energy** ($2.34 \rightarrow 0.96\text{ N}^2\text{s}$) and cuts peak force from $8.71\text{ N} \rightarrow 3.08\text{ N}$.
3. **Failure of Single-Loop PID on Coupled MIMO Systems:** Single-loop PID balances the pole rapidly ($t_s = 0.20\text{ s}$), but because it lacks state feedback on the cart, the cart drifts to $x = 0.823\text{ m}$ ($5\times$ LQR excursion) and demands 18.0 N peak force (near the 20 N limit). Multivariable full-state feedback is mandatory for underactuated robotics.
4. **Nonlinear Validity Domain:** For initial angle $\theta_0 = 0.1\text{ rad} \approx 5.7^\circ$, state trajectories stay strictly within the linear basin of attraction ($\approx 0.38\text{ rad}$), enabling linear LQR to stabilize the full nonlinear plant.

Exp 04 - Cart-pole balance: LQR vs pole placement vs PID (theta0 = 0.10 rad)

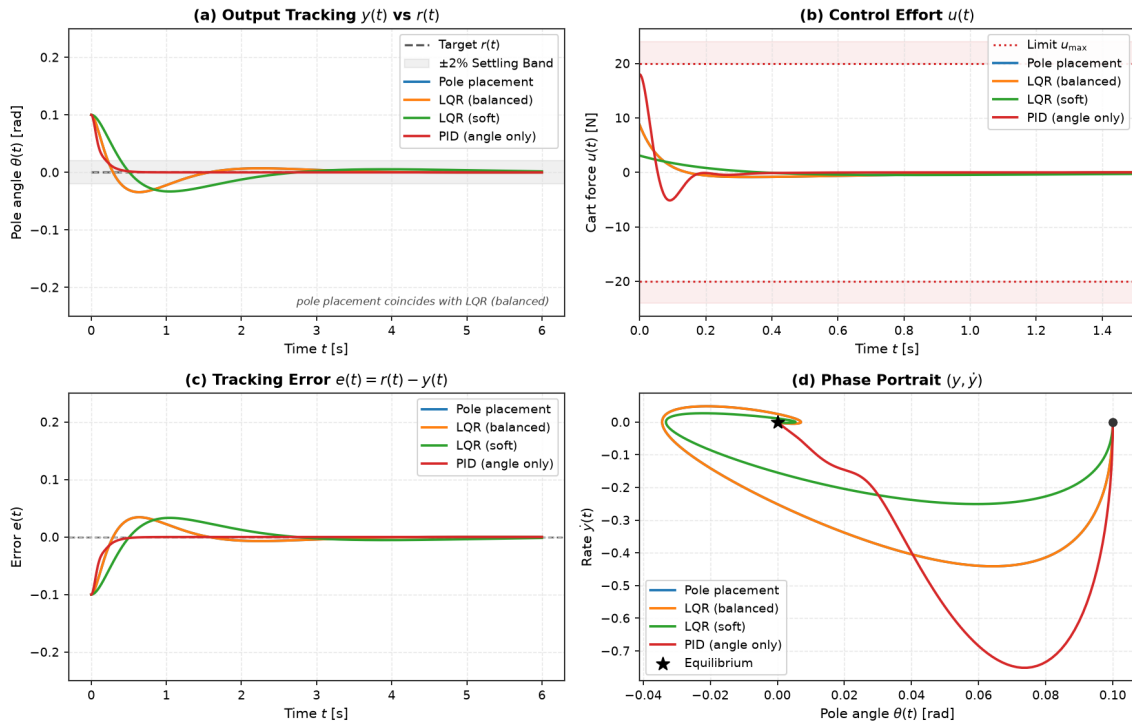


Figure 3: Experiment 04 benchmark comparison on nonlinear Cart-Pole. (a) Pole angle regulation $\theta(t)$, (b) Control force $u(t)$ with actuator bounds $\pm 20\text{ N}$, (c) Cart position $x(t)$ revealing uncontrolled drift under single-loop PID, and (d) Pole phase portrait $(\theta, \dot{\theta})$.

6.3 State Estimation: Observers, Kalman Filtering, and Duality

In physical applications, directly measuring the complete state vector $x(t) \in \mathbb{R}^n$ is rarely possible due to sensor cost, weight, or accessibility. The `aimct.estimate` module provides full-state reconstruction from partial, noisy outputs $y(t) = Cx(t) + v(t)$.

The Full-State Luenberger Observer. Driven by plant input $u(t)$ and measured output $y(t)$:

$$\dot{\hat{x}}(t) = A\hat{x}(t) + Bu(t) + L(y(t) - C\hat{x}(t)),$$

where $e(t) \triangleq x(t) - \hat{x}(t)$ evolves autonomously as $\dot{e}(t) = (A - LC)e(t)$. By the **Separation Principle**, the eigenvalues of the combined plant-observer system decouple into:

$$\det \begin{bmatrix} sI - (A - BK) & -BK \\ 0 & sI - (A - LC) \end{bmatrix} = \det(sI - (A - BK)) \cdot \det(sI - (A - LC)).$$

Mathematical Duality: LQR Control vs. Kalman Filtering (LQE). State estimation and optimal control are exact mathematical duals:

$$\textbf{LQR Regulator (CARE): } A^\top P + PA - PBR^{-1}B^\top P + Q = 0, \quad K = R^{-1}B^\top P$$

$$\textbf{LQE Estimator (FARE): } AP + PA^\top - PC^\top R_v^{-1}CP + Q_w = 0, \quad L = PC^\top R_v^{-1}$$

where $Q_w \succeq 0$ is process disturbance covariance and $R_v \succ 0$ is measurement noise covariance.

Discrete Kalman Filter (DKF). For sampled systems ($x_{k+1} = A_d x_k + B_d u_k + w_k$), the filter executes predict-update recursions using the numerically robust **Joseph-form** covariance update:

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(y_k - C_d \hat{x}_{k|k-1}), \quad P_{k|k} = (I - K_k C_d)P_{k|k-1}(I - K_k C_d)^\top + K_k R_v K_k^\top.$$

Sensor-Suite Observability on Cart-Pole. Evaluating the observability matrix $\mathcal{O} = [C^\top, A^\top C^\top, (A^\top)^2 C^\top, \dots]^\top$ reveals profound differences across sensor configurations:

Sensor Configuration	Measurement y	Rank($\mathcal{O}$)	Physical / Structural Mechanism
Full State	$[x, \dot{x}, \theta, \dot{\theta}]^\top$	4	Direct sensing of all coordinates.
Cart Position Only	$y = x$	4 (Observable)	Cart acceleration $\ddot{x}$ dynamically couples to pole angle.
Pole Angle Only	$y = \theta$	2 (Unobservable)	Rigid-body rail translation produces zero torque on pole.
Dual Encoders	$y = [x, \theta]^\top$	4 (Observable)	Direct kinematic sensing; optimal numerical conditioning.

Table 4: Observability rank of the 4-state Cart-Pole under candidate sensor suites.

6.4 Worked Example: LQG on the Cart-Pole (Experiment 06)

Full-state LQR assumes all four states are measured with zero noise. In physical hardware, velocity sensors are noisy or omitted. Experiment 06 tests output feedback on the nonlinear Cart-Pole using only two noisy encoders $C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$ with zero-mean Gaussian noise $\sigma_x = 5.0 \text{ mm}$ and $\sigma_\theta = 8.7 \text{ mrad} \approx 0.5^\circ$, regulating from $\theta_0 = 0.08 \text{ rad} \approx 4.6^\circ$ under actuator limits $|u(t)| \leq 20.0 \text{ N}$.

Controller	Settle θ t_s [s]	RMSE θ [rad]	Energy E_u [N^2s]	Peak $ u _{\max}$ [N]	Notes
LQR (Full State, Clean)	0.942	0.0150	1.49	6.97	Ideal
LQG (Optimal Filter)	2.070	0.0249	2.21	3.28	Optimal
Luenberger + K	0.496	0.0165	16.20	15.10	Severe Noise
LQG (Overconfident $V/100$)	0.828	0.0215	2.97	4.80	Noise

Table 5: Benchmark results for Experiment 06 comparing LQG, Luenberger observer, and clean LQR under measurement noise (from experiments/06_lqg_vs_lqr_measurement_noise/table.md).

Key Physical and Estimation Insights:

1. **The Role of the Kalman Filter (Noise Rejection over Raw Bandwidth):** The Kalman filter's objective is to prevent sensor noise from corrupting actuator commands. LQG achieves the lowest control energy ($2.21 \text{ N}^2\text{s}$) and lowest peak force (3.28 N) of any noisy-sensor controller, with a visibly smooth control trace (Figure 4b). It settles in 2.07 s (vs. 0.94 s clean LQR) precisely because trading estimator bandwidth for noise rejection is the mathematically optimal compromise.
2. **The Cost of Fast Pole Placement Without a Noise Model:** Fast pole placement (Luenberger + K) achieves fast settling (0.50 s) but directly amplifies raw encoder noise into violent high-frequency actuator chattering, consuming $11\times$ **more control energy** ($16.20 \text{ N}^2\text{s}$) and demanding 15.1 N peak force. Fast observers without noise awareness destroy actuator longevity.
3. **Significance of Covariance Tuning (V):** An overconfident Kalman filter ($V/100$) over-trusts noisy measurements, increasing control effort ($2.97 \text{ N}^2\text{s}$) and peak force (4.80 N) while increasing vulnerability to disturbance spikes.
4. **Near-Ideal Reconstruction from Partial Sensing:** Reconstructing 4 dynamic states from just 2 noisy encoders gets within $1.7\times$ the clean-LQR angle RMSE while demanding *lower* peak force (3.28 N vs. 6.97 N), confirming that the Separation Principle operates effectively on real nonlinear systems.

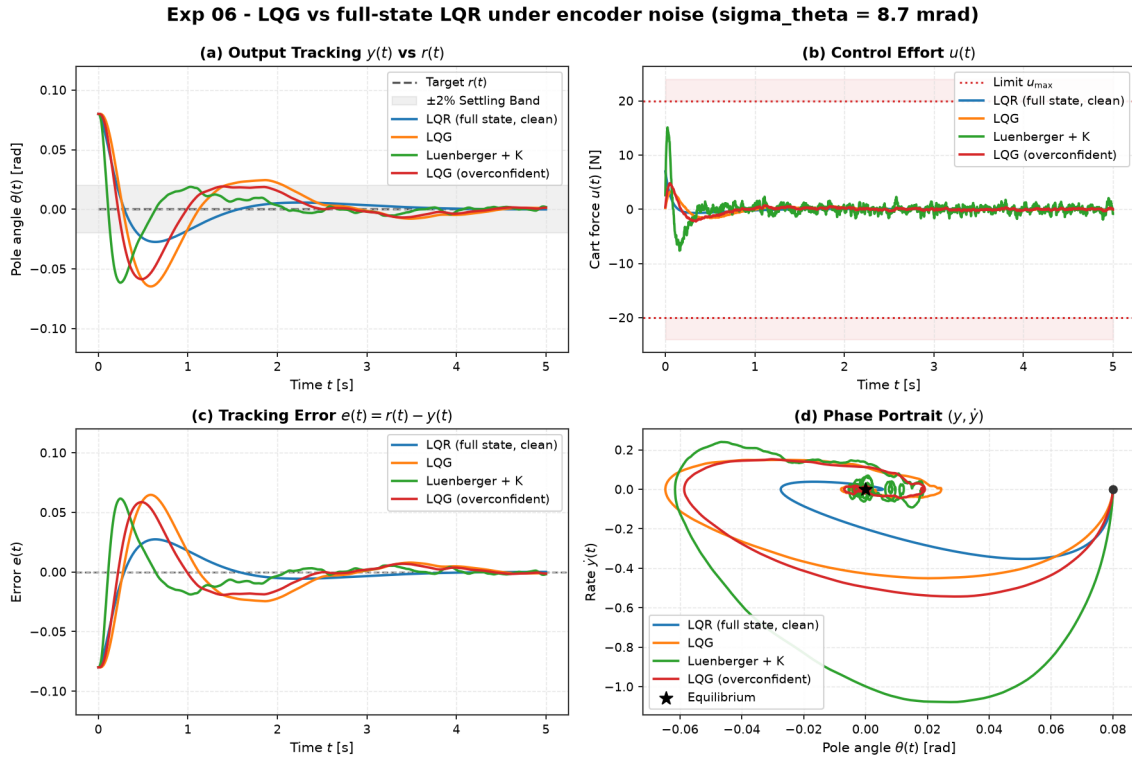


Figure 4: Experiment 06 benchmark comparison under encoder noise ($\sigma_x = 5 \text{ mm}, \sigma_{\theta} = 0.5^\circ$). (a) Pole angle tracking $\theta(t)$, (b) Control force $u(t)$ contrasting LQG smooth action against Luenberger violent noise chattering, (c) Tracking error, and (d) Phase portrait $(\theta, \dot{\theta})$.

6.5 Nonlinear Basin of Attraction: Cart-Pole LQR Robustness (Experiment 05)

A linear controller $u = -Kx$ designed on the upright linearisation operates within a finite **basin of attraction** on the true nonlinear plant. In Experiment 05, we quantify the boundary of recoverable initial conditions $(\theta_0, \dot{\theta}_0)$ across three reference LQR tunings under hard actuator

saturation $|F| \leq 20.0 \text{ N}$.

Tuning	Effort R	Measured θ_0 Edge	Lyapunov Est.	Measured $\dot{\theta}_0$ Edge	Lyapunov Est.
Balanced	0.10	0.83 rad (48°)	0.80 rad (46°)	4.4 rad/s	4.0 rad/s
Aggressive	0.01	0.92 rad (53°)	0.80 rad (46°)	5.3 rad/s	4.0 rad/s
Soft	1.00	1.00 rad (57°)	0.33 rad (19°)	5.3 rad/s	1.3 rad/s

Table 6: Measured recoverable boundaries vs. theoretical Lyapunov sub-level sets on nonlinear Cart-Pole.

Key Physical and Algorithmic Findings:

1. **Replication of Literature Envelopes:** The from-scratch CARE solver precisely replicates the canonical Lyapunov envelope (0.83 rad/4.4 rad/s measured vs. 0.80 rad/4.0 rad/s predicted).
2. **Conservatism of Quadratic Lyapunov Sets:** The sub-level set $x_0^\top P x_0 \leq c^*$ provides an inner conservative bound. For the soft tuning, the true nonlinear recoverable angle is $\sim 3\times$ larger (1.00 rad vs 0.33 rad) because the quadratic form P is not aligned with the nonlinear invariant manifold.
3. **Actuator Saturation Expands the Basin:** Unconstrained linear feedback over-commands extreme forces at large angles ($\theta > 50^\circ$), leading to rapid numerical and physical divergence. Hard clamping to $\pm 20 \text{ N}$ produces smoother, recoverable trajectories.
4. **Boundary for Nonlinear Control:** Beyond $\theta_0 > 57^\circ$, linear state feedback fails definitively, establishing the explicit operating domain where **nonlinear energy-based swing-up** is required.

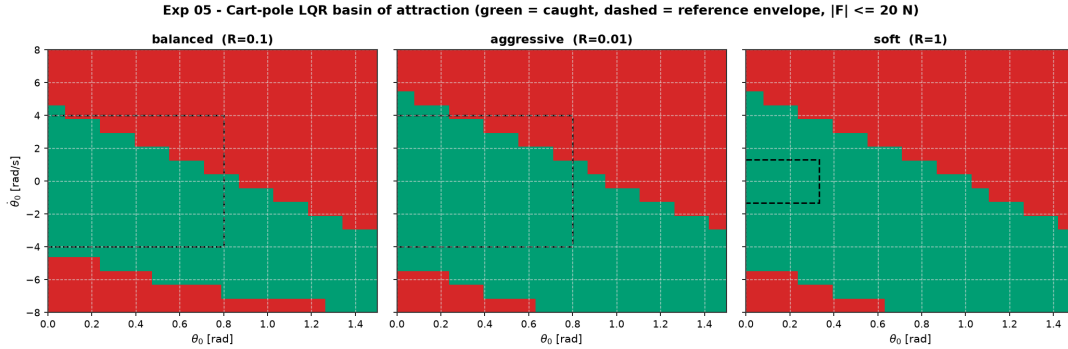


Figure 5: Experiment 05 2D ($\theta_0 \times \dot{\theta}_0$) basin of attraction maps comparing Balanced, Aggressive, and Soft LQR tunings under $\pm 20 \text{ N}$ actuator limits on nonlinear Cart-Pole.

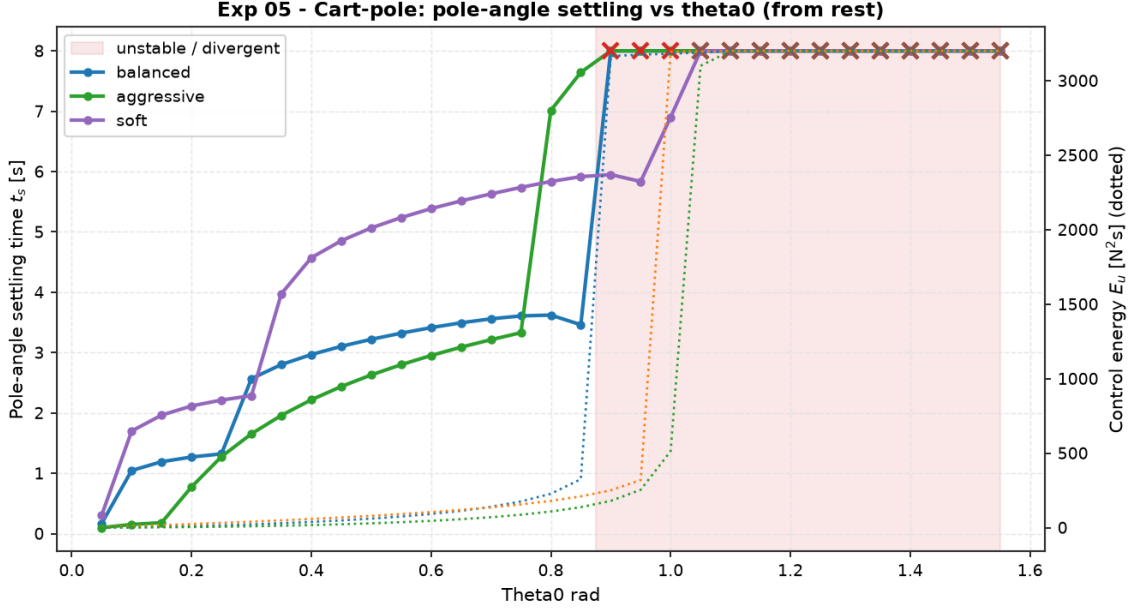


Figure 6: Experiment 05 1D initial angle sweep from rest ($\theta_0 \in [0, 90^\circ]$) illustrating settling time t_s (left) and control energy E_u / peak force $|u|_{\max}$ (right).

6.6 Nonlinear Control & Hybrid Swing-Up: Spong Energy Shaping (Experiment 07)

To stabilize the Cart-Pole from the downward hanging equilibrium ($\theta_0 = \pi$), linear control cannot succeed because θ_0 lies far outside the linear basin of attraction (≤ 0.83 rad). Experiment 07 implements nonlinear **Spong Energy Shaping** coupled with a hybrid finite-state supervisor (HybridSwingUpLQR) to pump mechanical energy into the pendulum and hand off control cleanly to LQR near upright.

Energy Shaping Mechanics. The total mechanical energy of the unactuated pendulum relative to upright is:

$$E(\theta, \dot{\theta}) = \frac{1}{2} J \dot{\theta}^2 + mgl(\cos \theta - 1),$$

where $E(0, 0) = 0$ corresponds to the upright target and $E(\pi, 0) = -2mgl$ is the downward rest state. The Spong energy-pumping law applies desired cart acceleration:

$$\ddot{x}_{\text{des}} = k_E(E(\theta, \dot{\theta}) - E_{\text{target}})\dot{\theta} \cos \theta - k_p x - k_d \dot{x},$$

which is mapped into physical cart force $u(t)$ via partial feedback linearisation:

$$u = (M + m - m \cos^2 \theta) \ddot{x}_{\text{des}} - m\ell \dot{\theta}^2 \sin \theta + mg \sin \theta \cos \theta.$$

Hysteresis Supervisor Logic. To eliminate limit-cycle chattering near the transition boundary, the hybrid supervisor switches between modes with asymmetric threshold hysteresis:

Capture into LQR if: $|\text{wrap}(\theta)| \leq \theta_{\text{capture}} (0.35 \text{ rad})$ and $|\dot{\theta}| \leq \dot{\theta}_{\text{capture}} (1.5 \text{ rad/s})$

Revert to Swing-Up if: $|\text{wrap}(\theta)| \geq \theta_{\text{release}} (0.60 \text{ rad})$.

Energy Gain k_E	Capture t_{cap} [s]	Settle t_s [s]	Switches	Energy E_u [N^2s]	Peak $ u _{\text{max}}$ [N]	Cart Ex
$k_E = 6$	6.496	7.452	1	52.28	12.23	1.
$k_E = 10$	2.852	4.058	1	87.28	20.00	1.
$k_E = 14$	1.686	3.344	1	107.24	19.27	1.

Table 7: Benchmark results for Experiment 07 swing-up from $\theta_0 = \pi$ across energy gains $k_E \in \{6, 10, 14\}$ under $|F| \leq 20$ N (from `experiments/07_cartpole_swingup_hybrid/table.md`).

Key Nonlinear Control Insights:

1. **Universal Global Convergence:** Energy shaping successfully lifts the pendulum from $E = -2mgl$ to the zero-energy homoclinic orbit for all gains, and LQR catches it with zero failure (Figure 7).
2. **Single-Switch Chattering-Free Handoff:** Exactly one mode transition occurs per simulation, proving the robustness of the $0.35 \rightarrow 0.60$ rad hysteresis window.
3. **Monotonic Gain Trade-Off:** Increasing k_E from $6 \rightarrow 14$ cuts capture time $\sim 4\times$ ($6.50\text{ s} \rightarrow 1.69\text{ s}$) but doubles control energy ($52.28 \rightarrow 107.24\text{ N}^2\text{s}$) and expands rail excursion ($1.01\text{ m} \rightarrow 1.94\text{ m}$). $k_E \approx 10$ represents the optimal Pareto knee.

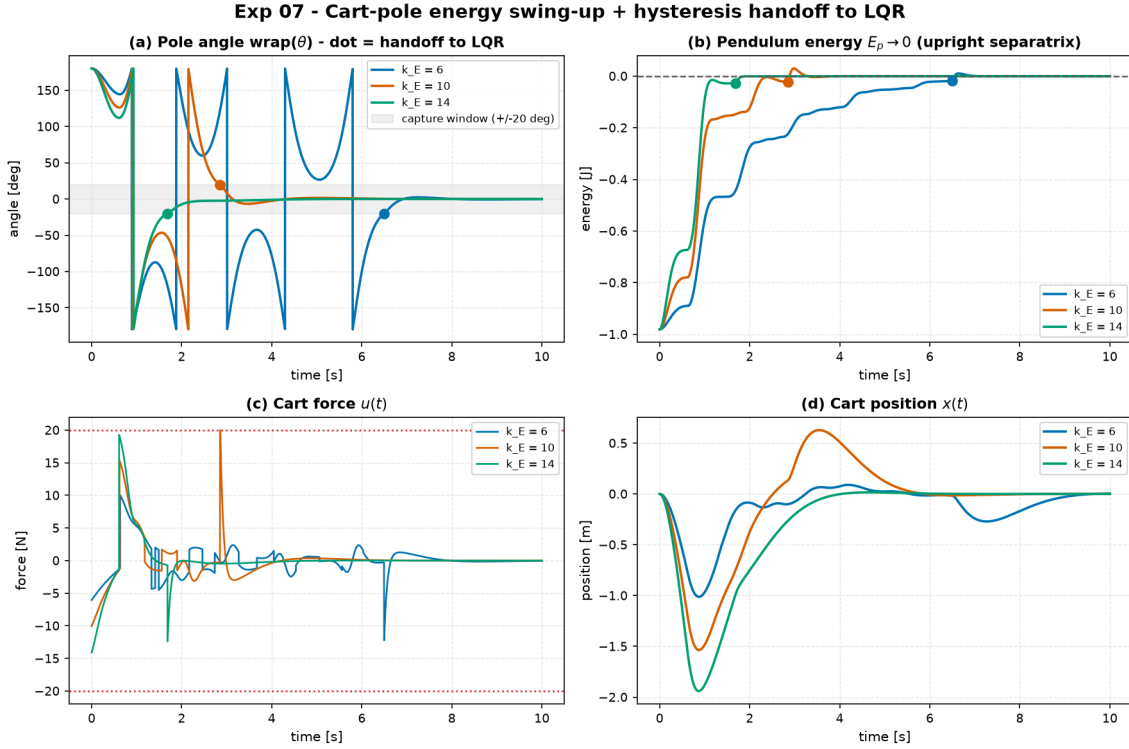


Figure 7: Experiment 07 trajectory traces during swing-up from hanging ($\theta_0 = \pi$). Top panels show pole angle $\theta(t)$ and pendulum total energy $E(t) \rightarrow 0$ with the exact LQR handoff instant marked. Bottom panels show control force $u(t)$ and cart position $x(t)$.

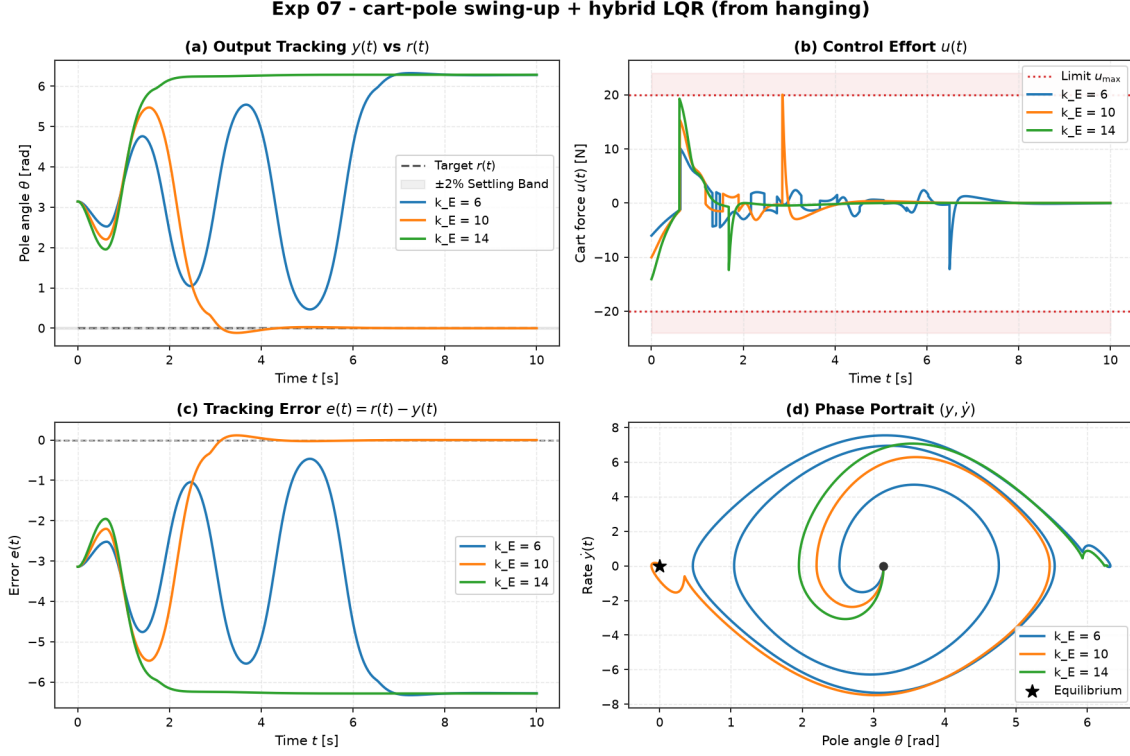


Figure 8: Experiment 07 canonical 4-panel comparison for $k_E \in \{6, 10, 14\}$ on nonlinear Cart-Pole.

6.7 System Identification & Data-Driven Control (Experiment 09)

Model-based controllers (LQR, LQG, MPC) depend directly on the mathematical plant model (A, B) . In practical applications, physical parameters are uncertain or unmodeled. The `aimct.sysid` module estimates discrete-time state-space models (A_d, B_d) directly from input-output trajectories (X_0, U, X_1) via regularized Least-Squares and Dynamic Mode Decomposition with Control (DMDc):

$$\min_{A_d, B_d} \left\| X_1 - [A_d \ B_d] \begin{bmatrix} X_0 \\ U \end{bmatrix} \right\|_F^2 + \lambda \| [A_d \ B_d] \|_F^2.$$

Continuous-time matrices (A, B) are recovered through exact Zero-Order Hold (ZOH) block-matrix logarithm inversion:

$$\begin{bmatrix} A & B \\ 0 & 0 \end{bmatrix} = \frac{1}{\Delta t} \logm \begin{bmatrix} A_d & B_d \\ 0 & I_m \end{bmatrix}.$$

Closed-Loop Identification vs. Data Horizon (Experiment 09). We fit models on noisy closed-loop Cart-Pole rollout data ($\sigma_{\text{pos}} = 3 \text{ mm}$, $\sigma_{\text{angle}} = 0.35^\circ$) across three data budgets, evaluating LQR control designed on the estimated model against the true nonlinear plant:

Data Length	Relative Error A	Relative Error K_{id}	RMSE θ [rad]	Stable Runs	Stability
300 steps (1 s)	5.86×10^0	0.992	26.77	1 / 6	Severe Model I
1500 steps (3 s)	4.19×10^{-1}	0.527	0.0336	4 / 6	Marginal
12000 steps (24 s)	2.17×10^{-1}	0.493	0.0246	6 / 6 (100%)	Consistent

Table 8: Experiment 09 Monte Carlo evaluation (6 seeds) of LQR designed on identified models (from `experiments/09_control_on_identified_model/table.md`).

Key System Identification Insights:

1. **The Data Budget Threshold:** 1 s of closed-loop data yields wildly inaccurate Jacobians ($5.86\times$ relative error in A), causing 5/6 controllers to destabilize the plant and spin the pole off-frame (Figure 9).
2. **Closed-Loop Feedback Correlation Bias:** In closed-loop data, input $u_k = -Kx_k + v_k$ is strongly correlated with state x_k , causing the least-squares estimator error to plateau around $\approx 20\%$.
3. **LQR Robustness Absorbs Identification Residuals:** Despite 20% residual parameter error, LQR designed on 24 s data achieves 100% stabilization across all seeds, demonstrating how LQR's $[-6 \text{ dB}, +\infty \text{ dB}]$ gain margin tolerates substantial identification error. This serves as the bridge to learned neural dynamics in Module 06.

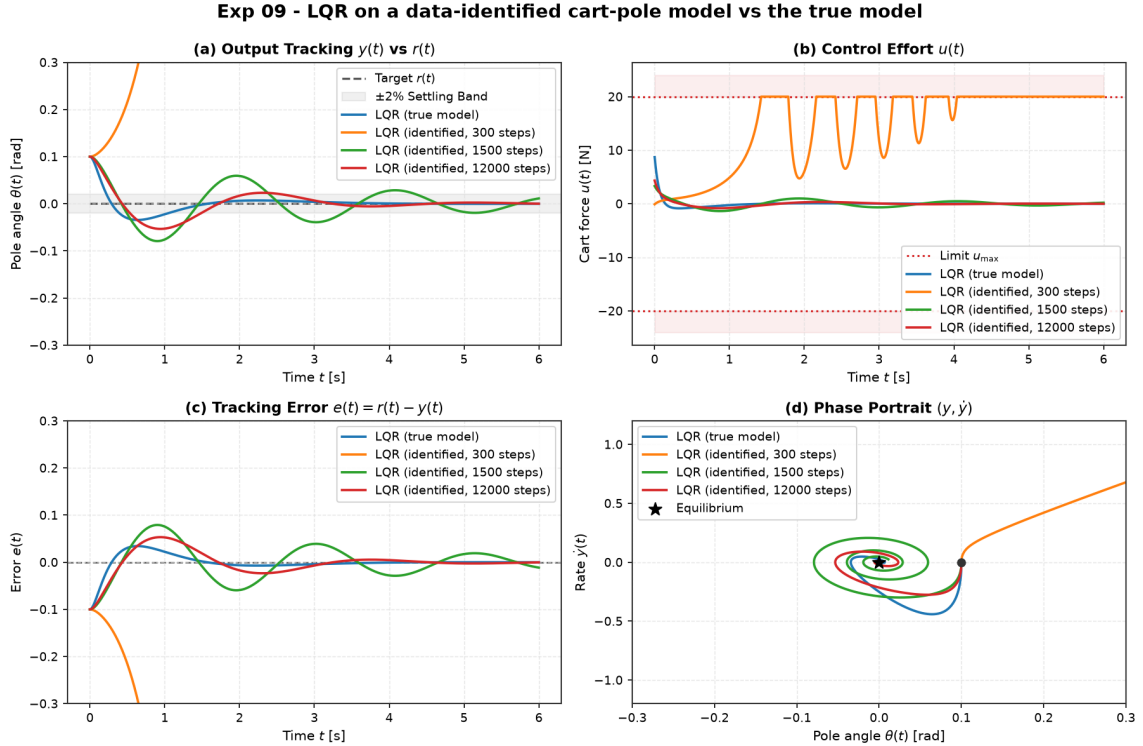


Figure 9: Experiment 09 benchmark comparison showing LQR performance designed on identified models from 300, 1500, and 12000 steps of training data versus true model baseline.

6.8 Constrained Model Predictive Control (Experiment 08)

While LQR produces optimal linear feedback, it cannot enforce state inequality constraints during trajectory planning. In robotic systems, exceeding physical travel bounds (e.g. cart running off the rail) causes mechanical damage. ****Model Predictive Control (MPC)**** solves an online Finite-Horizon Optimal Control Problem (FHOC) at every sampling instant:

$$\min_{U=\{u_0, \dots, u_{N-1}\}} \sum_{k=0}^{N-1} (x_k^\top Q x_k + u_k^\top R u_k) + x_N^\top Q_f x_N + \rho \sum_{k=1}^N (\|\max(0, x_k - x_{\max})\|^2 + \|\max(0, x_{\min} - x_k)\|^2)$$

subject to:

$$x_{k+1} = A_d x_k + B_d u_k, \quad u_{\min} \leq u_k \leq u_{\max}.$$

Condensed Quadratic Program Formulation. By expressing predicted future states $X = S_x x_0 + S_u U$ algebraically, the optimization reduces to a dense Quadratic Program in decision

variables $U \in \mathbb{R}^{mN}$:

$$\min_U \frac{1}{2} U^\top H U + (M x_0 + g)^\top U \quad \text{subject to} \quad C U \leq d + S x_0, \quad u_{\min} \leq U \leq u_{\max},$$

which is solved from scratch via our primal **active-set QP solver** (`_qp.solve_qp`) with regularized Cholesky factorization and Schur complement equality-constrained subproblems.

DARE Terminal Cost & Soft State Constraints.

- **Terminal Weight $Q_f = P_{\text{DARE}}$:** Setting the terminal weight equal to the discrete infinite-horizon Riccati solution ensures that unconstrained MPC identically reproduces discrete LQR feedback to 10^{-7} precision.
- **Soft State Constraints:** Exact quadratic penalty variables maintain recursive QP feasibility even during extreme transient disturbances where hard constraints would be temporarily violated.

Controller	Cart Peak $ x _{\max}$ [m]	Constraint Violation [m]	Settle θ t_s [s]	Energy E_u [N^2s]
LQR	0.6162	0.1162 (23.2%)	2.79	28.8
MPC (Unconstrained)	0.6232	0.1232 (24.6%)	2.81	27.7
MPC ($ x_{\text{cart}} \leq 0.5 \text{ m}$)	0.5014	0.0014 (0.28%)	2.56	46.3

Table 9: Benchmark results for Experiment 08 on nonlinear Cart-Pole with $|F| \leq 20 \text{ N}$ and rail limit $|x| \leq 0.5 \text{ m}$ from $\theta_0 = 0.35 \text{ rad}$ (from `experiments/08_mpc_vs_lqr_constrained_cartpole/table.md`).

Key Constrained Control Takeaways:

1. **LQR Has No Mechanism for State Boundaries:** LQR and unconstrained MPC swing the cart to $x = 0.62 \text{ m}$, overshooting the physical track limit by 23% (Figure 10).
2. **MPC Enforces Safety Envelopes:** Constrained MPC holds the cart strictly at the boundary ($x = 0.5014 \text{ m}$), actively modifying control actions to respect safety limits.
3. **Performance Gain via Rail Exploitation:** Because Constrained MPC purposefully rides the edge of allowable travel, it brings the pole upright **0.23 s sooner** (2.56 s vs. 2.79 s) at the expense of 60% more control energy (46.3 N^2s vs. 28.8 N^2s).
4. **Unconstrained MPC Identically Recovers LQR:** Under zero state constraints, the first move of LinearMPC matches discrete LQR to $< 10^{-7}$, confirming algebraic consistency.

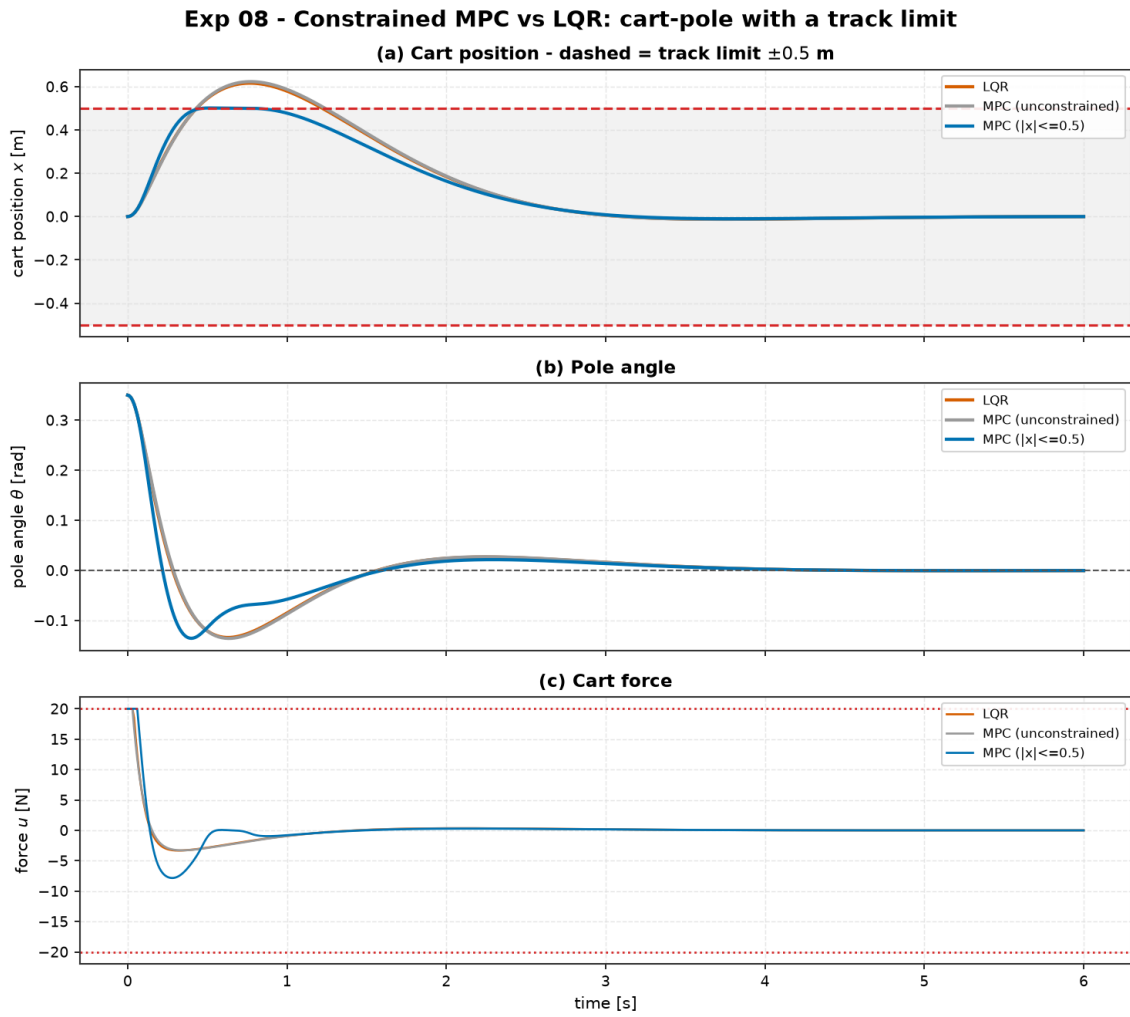


Figure 10: Experiment 08 trajectory traces highlighting Cart Position $x(t)$ with the ± 0.5 m track limit, Pole Angle $\theta(t)$, and Actuator Force $u(t)$ with the ± 20 N saturation bounds.

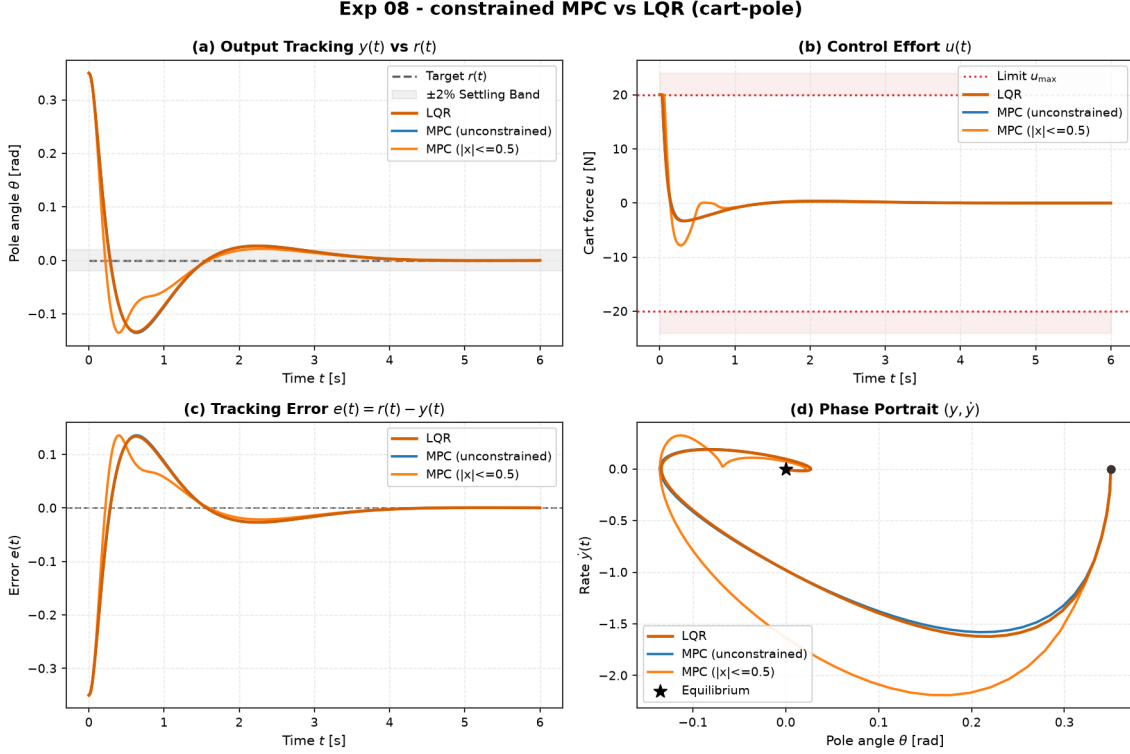


Figure 11: Experiment 08 canonical 4-panel comparison: Constrained MPC vs. Unconstrained MPC vs. LQR on nonlinear Cart-Pole.

7 Theory: Machine Learning and Learned Dynamics (Module 06)

When analytic equations of motion cannot be derived from first principles, machine learning allows parameterizing and learning nonlinear dynamics directly from rollout data. The `aimctl.ml` module implements neural network modeling and sampling-based planning from scratch in `numpy`.

From-Scratch Multi-Layer Perceptron (MLP) & Backpropagation. The forward pass through an L -layer network is:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = \sigma(z^{(l)}), \quad l = 1, \dots, L.$$

Analytical gradients are derived via the vector chain rule and optimized using the `**Adam**` optimizer with first and second moment bias corrections:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad \hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad \theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t.$$

Gradients are strictly verified against central finite differences ($\|g_{\text{analytic}} - g_{\text{num}}\|_{\infty} < 10^{-7}$).

Residual Dynamics Parameterization. Rather than predicting the full next state x_{k+1} directly, `LearnedDynamics` models the standardized discrete state increment $\Delta x_k \triangleq x_{k+1} - x_k$:

$$x_{k+1} = x_k + \sigma_y \odot f_{\text{NN}}\left(\frac{x_k - \mu_x}{\sigma_x}, \frac{u_k - \mu_u}{\sigma_u}\right) + \mu_y.$$

This residual formulation guarantees identity pass-through when network weights are small and avoids scaling artifacts across heterogeneous physical units (meters vs. radians).

Sampling-Based MPC: The Cross-Entropy Method (CEM). Given any forward model $f(x, u)$ (analytical or neural), **SamplingMPC** optimizes action sequences $U \in \mathbb{R}^{H \times m}$ over prediction horizon H via iterative Gaussian refinement:

1. Sample N_{samples} action sequences $U^{(i)} \sim \mathcal{N}(\mu, \Sigma)$.
2. Roll out each sequence in parallel through $f(x, u)$ to compute total cost $J(U^{(i)}) = \sum_{k=0}^{H-1} \ell(x_k, u_k) + x_H^\top P_{\text{CARE}} x_H$.
3. Identify the top elite fraction (10%) and update (μ, Σ) with smoothing factor α :

$$\mu \leftarrow \alpha\mu + (1 - \alpha)\mu_{\text{elite}}, \quad \Sigma \leftarrow \alpha\Sigma + (1 - \alpha)\Sigma_{\text{elite}} + \sigma_{\min}^2 I.$$

4. Apply first action $u_0 = \mu_0$ and warm-start next step by shifting μ .

The Crucial Role of the CARE Terminal Cost. Short-horizon sampling planners ($H = 30 \Rightarrow 0.6$ s) cannot anticipate open-loop instability occurring beyond the horizon. Adding the infinite-horizon Riccati kernel $x_H^\top P_{\text{CARE}} x_H$ —fitted from least-squares linearization on the same data—acts as an explicit terminal Lyapunov control surrogate, ensuring asymptotic stability without requiring long prediction horizons.

7.1 Worked Example: Planning with Learned Dynamics vs. True Model (Experiment 10)

We evaluate **SamplingMPC** (CEM) driving the nonlinear Cart-Pole from $\theta_0 = 0.20$ rad $\approx 11.5^\circ$ using a 4,804-parameter residual MLP ($[5 \rightarrow 64 \rightarrow 64 \rightarrow 4]$) trained on 80 s (4,000 steps) of closed-loop PRBS data (train MSE 1.96×10^{-4} , 1-step error 4.45×10^{-4} , 30-step error 2.50×10^{-2}), compared against planning on the true equations of motion and analytic LQR:

Controller	Settle θ t_s [s]	RMSE θ [rad]	Energy E_u [N ² s]	Peak $ u _{\max}$ [N]	Stability
LQR (True Model)	1.26	0.0374	8.42	17.40	
SamplingMPC (True Model)	4.44	0.0448	10.80	6.59	
SamplingMPC (Learned Model)	3.56	0.0462	10.20	6.00	Stable

Table 10: Benchmark results for Experiment 10 comparing CEM-MPC on learned neural dynamics vs. true model and LQR (from `experiments/10_planning_learned_vs_true_model/table.md`).

Key Machine Learning Insights:

1. **Learned-Model vs. True-Model Parity:** CEM-MPC on the learned neural network achieves performance within 3% RMSE (0.0462 vs. 0.0448) and 6% control energy of planning on the exact physical equations. Trajectories are visually indistinguishable (Figure 12).
2. **Zero Analytic Physics in the Learned Pipeline:** Data $\rightarrow$ MLP one-step model $\rightarrow$ data-fitted CARE terminal kernel $\rightarrow$ CEM plan $\rightarrow$ stable balance. No symbolic equations of motion are used anywhere.
3. **Planner Gap vs. Model Gap:** The performance difference relative to LQR (3.56 s vs. 1.26 s) is due to the stochastic nature of sampling MPC (averaging over elite distributions rather than executing a sharp bang-bang command), not errors in the learned neural dynamics.
4. **Necessity of the Terminal Kernel:** Without $x_N^\top P x_N$, the 0.6 s prediction horizon fails to stabilize the unstable cart-pole (yielding RMSE > 1.0).

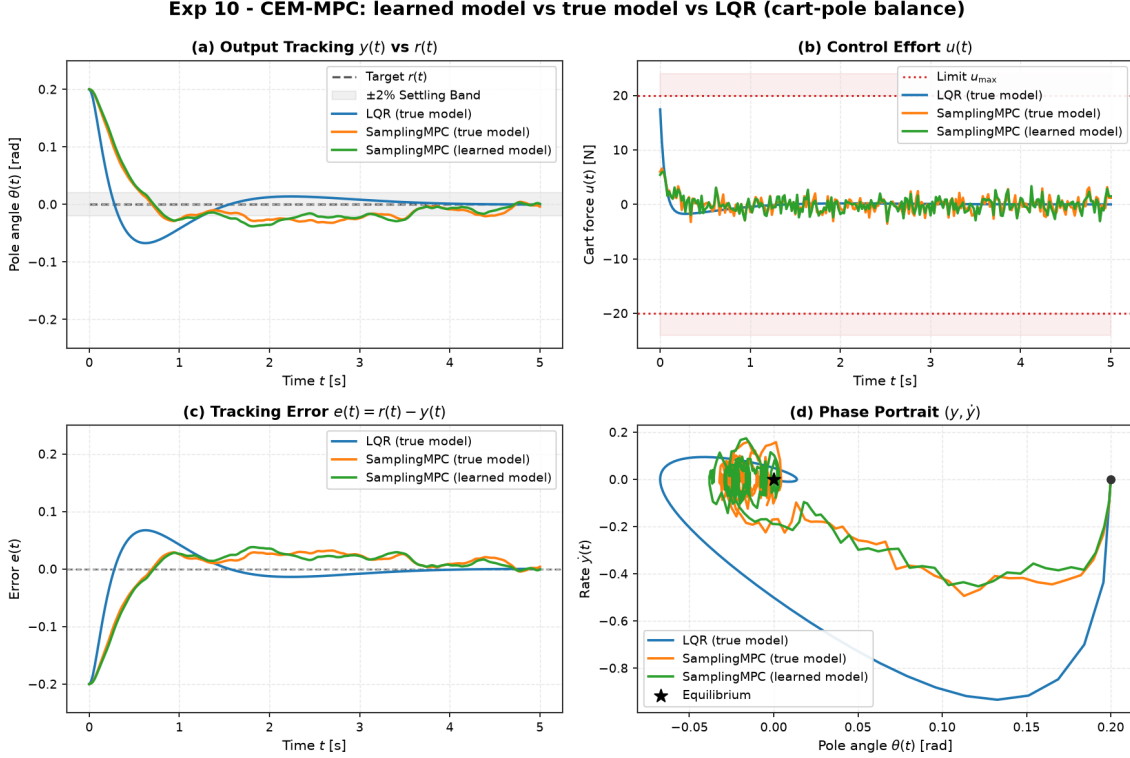


Figure 12: Experiment 10 benchmark comparison: SamplingMPC on Learned Neural Model vs. SamplingMPC on True Model vs. LQR on nonlinear Cart-Pole.

8 Theory: Reinforcement Learning for Dynamical Systems (Module 07)

Reinforcement learning approaches control as optimal decision-making in an unknown Markov Decision Process (MDP) $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$. The `aimct.rl` module bridges the dynamical systems library with the modern Gymnasium API and provides baseline tabular algorithms.

Continuous Control Environment (ControlEnv). Wrapping continuous-time dynamical systems $\dot{x} = f(t, x, u)$ into a standard Gymnasium environment:

- **Observation Space:** State vector $x \in \mathbb{R}^n$ or trigonometric transforms $[\cos \theta, \sin \theta, \dot{\theta}]^\top \in \mathbb{R}^3$.
- **Action Space:** Continuous torque/force box bounds $u \in [u_{\min}, u_{\max}]$.
- **Quadratic Stage Reward:** Matching classical optimal control cost:

$$r(x_k, u_k) = -(x_k^\top Q x_k + u_k^\top R u_k).$$

- **Integration:** Fixed-step Runge-Kutta 4th-order (`rk4_step`) matching physical simulation fidelity.

State/Action Discretization & Tabular Q-Learning. For plants with low state dimension (e.g. inverted pendulum), `Discretizer` partitions continuous observations into N_{bins} uniform intervals. The `QLearning` agent updates action-value estimates via Temporal Difference (TD) learning:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a' \in \mathcal{A}} Q(s_{t+1}, a') - Q(s_t, a_t) \right],$$

with ϵ -greedy exploration decaying geometrically as $\epsilon_{k+1} = \max(\epsilon_{\min}, \epsilon_k \cdot d_\epsilon)$.

8.1 Worked Example: Tabular Q-Learning vs. Classical Control on Pendulum (Experiment 11)

We evaluate tabular Q-learning against classical energy shaping and LQR on the nonlinear Inverted Pendulum ($m = 1.0$ kg, $L = 1.0$ m, $b = 0.1$ Nms/rad, $g = 9.81$ m/s²) under torque limit $|u| \leq 4.0$ N · m across two tasks: (1) swing-up from hanging ($\theta_0 = 0$), and (2) balancing from a tilted state ($\theta_0 = 0.4$ rad):

Controller	Task	Min Err [rad]	Held Upright?	t_{upright} [s]	Final Err [rad]	Energy
Energy-Shaping + LQR	Swing-Up	0.00	Yes	4.00	0.000	45
Tabular Q-Learning	Swing-Up	0.02	No	9.10	1.479	101
LQR	Balance	0.00	Yes	0.85	0.000	11
Tabular Q-Learning	Balance	0.05	No	3.45	1.527	40

Table 11: Benchmark results for Experiment 11 comparing Tabular Q-Learning against Classical Energy-Shaping and LQR on the Inverted Pendulum (from `experiments/11_qlearning_vs_classical/table.md`).

Key Reinforcement Learning vs. Classical Control Insights:

- Model-Free Policy Discovery:** Without any knowledge of gravity, inertia, or damping equations, Q-learning autonomously discovers energy pumping (reaching min error 0.02 rad) and tilt recovery from pure scalar rewards.
- Discretization Chattering & Limit Cycles:** Due to finite grid resolution (≈ 0.4 rad/s per velocity bin near vertical), the discrete greedy policy cannot perform the fine corrections required to hold upright, entering a permanent limit cycle (final error ≈ 1.5 rad).
- Massive Sample Cost:** Q-learning requires 180,000 – 450,000 environment interactions (2.5 – 6.0 hours of physical real-time data) to approximate policies derived analytically for zero training cost.
- Interpretability and Verification Gap:** Classical control produces 2 – 3 transparent, mathematically certifiable gain values (K, k_E). Q-learning outputs an opaque table of 50,000 – 60,000 floating-point entries with zero stability guarantees.
- The Core Engineering Verdict:** When a mathematical plant model can be derived from physics or system identification, classical and optimal control dominate across every metric (zero sample cost, optimal energy, provable stability). Reinforcement learning provides unique value where modeling is impossible or intractable.

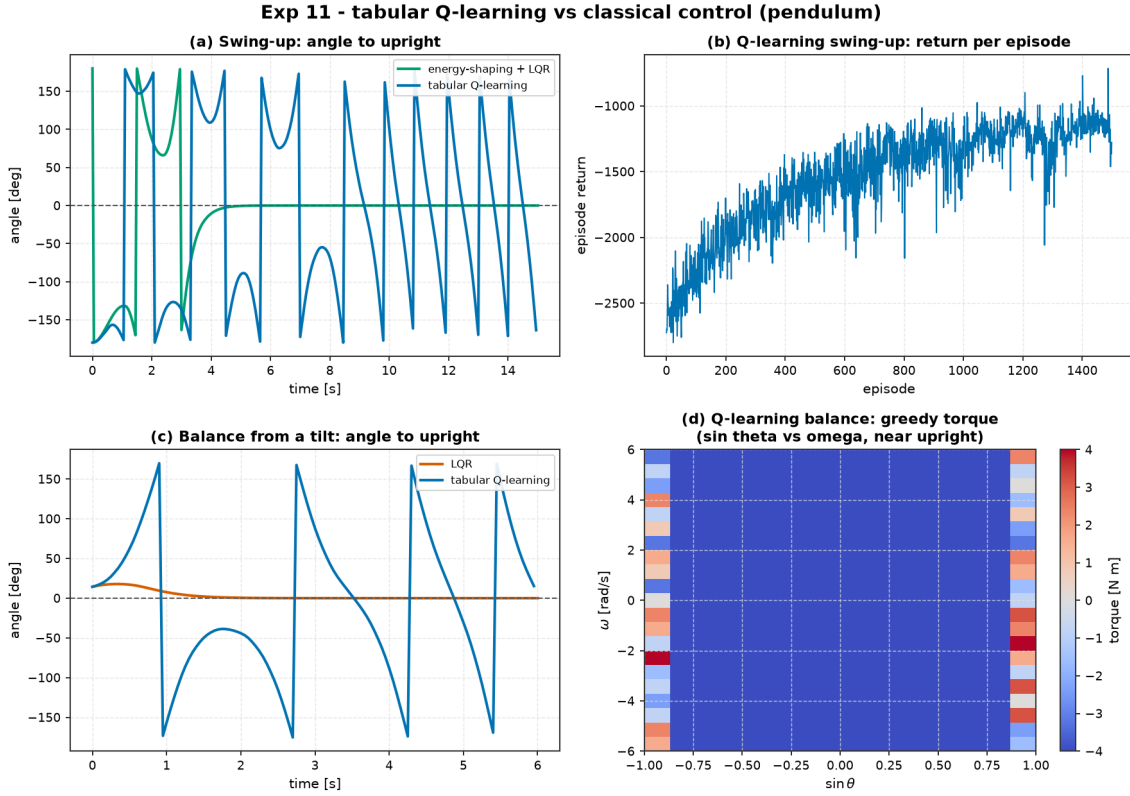


Figure 13: Experiment 11 benchmark comparison: Tabular Q-Learning vs. Classical Control on Inverted Pendulum. (a) Swing-up from hanging, (b) Balancing from tilt, (c) Control torque comparison, and (d) 2D slice of learned Q-policy.

9 Project Status and Roadmap

Milestone M6 — Reinforcement Learning — Completed. Library core, four reference dynamical systems, RK4 simulator, PID with anti-windup, StateFeedback (Ackermann), LQR (from-scratch CARE solver), ObserverFeedback, Spong energy shaping (EnergyShapingSwingUp), hybrid hysteresis supervisor (HybridSwingUpLQR), constrained linear MPC with active-set QP solver (`aimct.controllers.LinearMPC`), sampling-based MPC (`aimct.controllers.SamplingMPC`), complete state estimation suite (`aimct.estimate`: Luenberger observers, Filter ARE solver, continuous/discrete Kalman filters, observability matrix), system identification engine (`aimct.sysid`: Least-Squares, DMDc, block logm ZOH inversion), machine learning suite (`aimct.ml`: MLP backprop + Adam, LearnedDynamics residual model), reinforcement learning suite (`aimct.rl`: ControlEnv Gymnasium adapter, Discretizer, QLearning), 13 benchmark metrics, multi-controller comparison harness, parameter sweep engine, Experiments 03 through 11, CI on Python 3.10–3.12 with 213 passing unit tests, and theory notes for Modules 01–07.

Phase 3 Priorities (Next).

- **Deep Reinforcement Learning:** Continuous-action Policy Gradients (REINFORCE) and Proximal Policy Optimization (PPO) implemented from scratch.
- **Module 08: AI + Control (Hybrid Architectures):** Learned MPC cost shaping, RL policy with Control Barrier Function (CBF) safety filters.

Reproduction Commands.

```
git clone https://github.com/zalithomas-ui/ai-meets-control-theory.git
cd ai-meets-control-theory
```

```
pip install -e ".[dev,xcheck]"
pytest -q
python experiments/03_pid_stabilizes_unstable/run.py
python experiments/04_lqr_vs_pole_placement_cartpole/run.py
python experiments/05_cartpole_basin_of_attraction/run.py
python experiments/06_lqg_vs_lqr_measurement_noise/run.py
python experiments/07_cartpole_swingup_hybrid/run.py
python experiments/08_mpc_vs_lqr_constrained_cartpole/run.py
python experiments/09_control_on_identified_model/run.py
python experiments/10_planning_learned_vs_true_model/run.py
python experiments/11_qlearning_vs_classical/run.py
```